

AUTOMATED SATELLITE SAFE MODE RECOVERY & BEHAVIOURAL DIAGNOSTICS

Project Technical Documentation

Hackathon Edition · Version 1.0 · June 2026

SECTION 1 — THE PROBLEM STATEMENT

1.1 Setting the Scene (For Complete Beginners)

Imagine a spacecraft as a flying computer the size of a bus or even a small car, hurtling through space at 28,000 km/h. It is surrounded by hard vacuum, bathed in intense radiation, and experiences temperature swings from -120°C in shadow to $+180^{\circ}\text{C}$ in direct sunlight. There is no one on board to fix anything, and calling for help involves waiting minutes — sometimes hours — for a radio signal to travel to Earth and back.

This spacecraft is controlled by thousands of sensors, computers, motors, and software programs that must all work together perfectly. When something goes wrong — even one small thing — the spacecraft faces a choice: keep going and risk catastrophic damage, or enter what is called

SAFE MODE — an emergency survival state.

1.2 What is Safe Mode?

Safe mode (also called "safing" or "contingency mode") is the spacecraft's built-in self-preservation reflex. Think of it as the satellite equivalent of a human going into shock — the body shuts down everything except the most critical functions to stay alive.

When safe mode activates, the spacecraft automatically does the following:

- Turns off ALL scientific instruments (cameras, spectrometers, radars — everything that was doing useful science work)
- Points its solar panels directly at the Sun — because without power, it dies
- Switches to its lowest power radio transmitter — a weak, slow-speed "emergency beacon" so it can still hear Earth
- Runs only its most basic survival software — attitude control and thermal management
- Stores a crash report (called a crash dump) of everything that was happening just before the fault
- Waits. And waits. Doing nothing useful, bleeding money, until human engineers on the ground diagnose and fix the problem.

Real-World Cost

NASA's MAVEN spacecraft spent 3 months in safe mode after an IMU sensor failure in 2022. ESA's SOHO satellite nearly died permanently in 1998 — it took 4 months to recover. Hubble has entered safe mode more than 20 times. Each incident costs millions of dollars in lost data and engineering time.

1.3 The Structural Problem in Detail

The safe mode recovery process, as it exists today, has **FOUR** compounding bottlenecks:

Bottleneck 1: The Haystack Problem

A modern satellite monitors over 32,000 individual data channels simultaneously (voltages, temperatures, spin rates, software counters, error registers...). When something goes wrong, ALL of this data is dumped into a crash log. Finding which one of those 32,000 parameters caused the fault is like finding one specific piece of hay in a haystack, in the dark, with no flashlight.

Bottleneck 2: The Speed-of-Light Prison

Radio signals travel at the speed of light — 299,792 km/s. Even at that speed, it takes 20 minutes to reach Mars. Every diagnostic test requires: send command → wait for round-trip delay → see result → think → send next command. For deep space missions, one wrong guess burns an entire day.

Bottleneck 3: Brittle Rule-Based Systems

Today's fault detection software uses "if temperature > 85°C, then alert" style threshold rules, decided 5–10 years before launch when the spacecraft was being designed. These rules cannot handle:

- Novel fault combinations never imagined at design time
- Subtle gradual degradation that stays within threshold limits until catastrophic failure
- Multi-subsystem cascade failures where multiple systems fail in sequence

Bottleneck 4: No Automatic Recovery Exists

ESA's official FDIR (Fault Detection, Isolation and Recovery) specification explicitly states: "Safe mode recovery requires ground interaction. No auto-recovery from safe mode is foreseen." That single sentence is the open problem you are building a solution for.

Project Goal

Build an AI agent that, when safe mode triggers, autonomously reads the crash dump, identifies the root cause subsystem, constructs a causal explanation tree, and safely executes step-by-step recovery commands — without any human involvement, and without waiting for Earth to respond.

SECTION 2 — HOW A SATELLITE WORKS (From Scratch)

2.1 The Six Major Subsystems

Every satellite, from a tiny CubeSat to the Hubble Space Telescope, is built from six main subsystems. Understanding these is essential, because your AI agent must understand what each one does and how they fail.

Subsystem	What It Does	Key Components
ADCS Attitude & Orbit Control	The 'steering' system. Keeps the satellite pointing in the right direction.	Gyroscopes, star trackers, reaction wheels, thrusters
EPS Electrical Power System	The 'battery + solar panel' system. Generates, stores, and distributes electricity.	Solar arrays, battery packs, power regulators, voltage sensors
OBC Onboard Computer	The 'brain' of the satellite. Runs all flight software and monitors all other systems.	Flight computer CPUs, memory, watchdog timers, software
COMMS Communications	The 'radio' system. Sends data to Earth and receives commands.	High-gain antennas, transponders, RF amplifiers
TCS Thermal Control System	The 'temperature regulator'. Keeps everything from freezing or melting.	Heaters, radiators, heat pipes, thermistors
PYLD Payload	The 'science instruments'. What the satellite is actually there to do.	Cameras, spectrometers, radar, GPS receivers

2.2 What is Telemetry?

Telemetry is the stream of data that a spacecraft continuously transmits back to Earth. It is the equivalent of a patient's medical chart in a hospital — a continuous record of every vital sign.

Every few seconds, thousands of sensors on board measure their local conditions and package those measurements into a data packet. These packets are downlinked (transmitted to Earth) where they are stored in databases for engineers to monitor. The most critical parameters are shown on dashboards in real-time at mission control.

2.3 The Telemetry Parameters Your AI Will Use

The following table lists the key telemetry parameter groups your system must monitor and understand. These are what your AI model reads when diagnosing a safe mode event.

Parameter Name	Subsystem	Nominal Range & Anomaly Indicator
Battery Voltage (V_bat)	EPS	28.0 – 33.6 V (28V bus) Critical low: < 22V
State of Charge (%)	EPS	20 – 100% Critical: < 15%
Solar Array Current (I_sa)	EPS	0 – 12 A depending on sun angle Anomaly: sudden drop to 0A in sunlight
Bus Voltage (V_bus)	EPS	28V ± 5% (26.6 – 29.4V) Out-of-range triggers power fault
Heater Power Draw (W)	EPS	Varies by zone Anomaly: unexpected spike = thermal runaway risk
Reaction Wheel Speed (rpm)	ADCS	–6000 to +6000 rpm Anomaly: stuck at 0 or saturated (max speed)
Gyroscope Rate (deg/s)	ADCS	0 – 7 deg/s in normal ops NaN or constant value = sensor failure
Star Tracker Status (flag)	ADCS	0=OK, 1=blinded by moon/sun, 2=fault Critical if all trackers = fault
Sun Sensor Angle (deg)	ADCS	0 – 180 deg from boresight Safe mode triggered if > 90 deg prolonged

Attitude Error (deg)	ADCS	< 0.01 deg in nominal pointing Anomaly: > 5 deg sustained
OBC Temperature (°C)	OBC	−10 to +60°C operating range Anomaly: > 70°C or < −15°C
CPU Load (%)	OBC	< 70% nominal Anomaly: sustained 100% → software loop
Memory Usage (MB)	OBC	< 80% RAM used Anomaly: monotonic increase → memory leak
Watchdog Counter	OBC	Resets every N seconds Anomaly: overflow = software thread hung
SEU Counter	OBC	0 in normal low-radiation orbit Anomaly: spike = cosmic ray hit
Fault Register (bitmask)	OBC	0x00000000 = no fault Bits 1–32 map to specific fault types
Safe Mode Entry Count	OBC	Cumulative counter Recurring entries = systemic problem
Transponder Lock Status	COMMS	1=locked, 0=not locked Loss = comms blackout
Signal-to-Noise Ratio (dB)	COMMS	> 10 dB nominal

		Anomaly: drops suggest antenna pointing issue
Component Temperatures (°C)	TCS	Per-component operating ranges Anomaly: exceeding survival limits
Heater Enable Flags	TCS	Binary on/off per zone Anomaly: heater on when it should be off

2.4 How Parameters Relate to Each Other (The Key Insight)

Individual parameters rarely tell the full story. The real diagnostic power comes from understanding the RELATIONSHIPS between parameters. This is exactly what an LLM-based causal reasoning system excels at. Here are the most important cross-system relationships:

Telemetry Observation	Causal Chain → Root Cause
If solar current (I_sa) drops suddenly to 0 in sunlight...	→ Satellite enters negative power budget → battery drains → V_bat drops → EPS fault → safe mode triggered. Root cause: solar array failure or eclipse miscount.
If gyroscope reads constant value (stuck) or NaN...	→ ADCS loses attitude knowledge → sun acquisition fails → solar panels lose sun pointing → power loss → thermal runaway. Root cause: SEU in gyro processor or gyro hardware failure.
If OBC CPU load hits 100% and watchdog overflows...	→ Software thread stuck in infinite loop → watchdog timeout → automatic computer reboot → safe mode. Root cause: software bug triggered by unusual data pattern.
If battery State-of-Charge drops below 20%...	→ EPS automatically sheds (cuts power to) non-critical loads → payload off → ADCS minimal mode → if SoC keeps falling, safe mode triggered. Root cause: power generation insufficient for load.
If attitude error exceeds 5 degrees for >30 seconds...	→ Sun sensor sees angle >90° → safe mode logic fires → all instruments off → sun acquisition mode. Root cause: reaction wheel failure, thruster misfire, or gyro fault.

SECTION 3 — HOW SAFE MODE IS HANDLED TODAY

3.1 The Manual Recovery Process (Step by Step)

This is exactly what a team of spacecraft engineers does today when their satellite enters safe mode. Understanding this process is crucial because your AI system is replacing it.

1. **ALERT:** Pagers, phones, and control room alarms go off. A safe mode entry alert fires automatically. Engineers are called in 24/7, sometimes woken at 3am.
2. **WAIT FOR CONTACT WINDOW:** Depending on the satellite's orbit, engineers may need to wait hours for the satellite to be visible from a ground station antenna. During this wait, they can do nothing.

3. **ESTABLISH COMMUNICATIONS:** Once in contact, the team first verifies the satellite is alive — checking for the emergency beacon signal on the low-gain antenna. This alone can take 30–90 minutes.
4. **DOWNLOAD CRASH DUMP:** The crash dump (thousands of lines of parameter values, event logs, fault register states) is downlinked at the slow safe-mode data rate. For a large satellite, this can take hours.
5. **MANUAL LOG ANALYSIS:** Engineers sit down with printed or digital logs and begin reading — line by line — through thousands of timestamped records. They are looking for the first anomalous reading that preceded the fault. This is the biggest bottleneck, taking 1–3 days for complex anomalies.
6. **HYPOTHESIS FORMATION:** Based on what they find, engineers form a hypothesis about the root cause. They cross-reference this with years of design documents, flight software specifications, and institutional memory.
7. **RECOVERY COMMAND UPLINK:** Engineers write a recovery script (a sequence of commands to restart specific modules). They review it multiple times, then uplink it. They wait for the round-trip delay.
8. **OBSERVATION:** They watch the resulting telemetry to see if the recovery worked. If not, return to step 5 and form a new hypothesis. Each wrong guess costs another full cycle.
9. **INCREMENTAL RESTORATION:** Once the immediate fault is resolved, each instrument and subsystem is brought back online one at a time, with checks after each step. This takes additional days.



Real Timeline

NASA's New Horizons: 3 days recovery (simple command error, but 10.5hr one-way comms delay). ESA SOHO: 4 months recovery (attitude loss + contact loss, complex multi-system failure). ESA MAVEN Mars: 3 months recovery (IMU hardware failure requiring full software workaround).

3.2 The FDIR Framework — Today's Technical Approach

FDIR stands for Fault Detection, Isolation, and Recovery. It is the onboard software framework that monitors the spacecraft and triggers safe mode. Understanding FDIR is critical because your project is building an AI-enhanced FDIR system.

FDIR Phase	What It Does Today	How It Works / Limitation
Fault Detection	Monitors all telemetry parameters against pre-defined thresholds. Example: if V_bat < 22V, raise power fault flag. Limitation: cannot detect subtle anomalies within nominal range.	Simple threshold comparisons. No learning. Very reliable but brittle.
Fault Isolation	Once a fault flag is raised, determines which subsystem caused it. Example: power fault → check solar array, battery, and load sequentially. Limitation: complex cascades confuse the isolation logic.	Rule-based decision trees written by engineers at design time.
Recovery	Executes a pre-planned response to a detected+isolated fault. Example: solar array fault → switch to backup array → check voltage. Limitation: ONLY handles failures anticipated during design. Novel faults = no recovery script → safe mode.	Hard-coded command sequences. Zero adaptability to new failure types.

3.3 The Gap Your Project Fills

The table below maps the gaps in today's approach to the capabilities your AI system provides:

Current Limitation	Your AI System's Solution
Manual log analysis takes 1–3 days	Your LLM reads and interprets crash dumps in seconds
FDIR only handles anticipated failures	Your LLM can reason about novel fault combinations using contextual knowledge
Root-cause isolation requires expert knowledge	Your causal inference engine builds fault trees automatically from parameter correlations
One wrong recovery guess = another full day lost	Your agent iterates through hypotheses in seconds, not days
No auto-recovery from safe mode exists (ESA official spec)	Your system is the first attempt to automate this — your key novelty claim
Recovery scripts require manual authoring by engineers	Your agent generates recovery command sequences from reasoning

SECTION 4 — WHAT YOUR AI MODEL LEARNS (Fine-Tuning)

4.1 What Does "Fine-Tuning" Actually Mean?

A large language model (like Phi-3 or Llama-3) is pre-trained on billions of web pages, books, and code. It already knows English, basic physics, and some spacecraft concepts. But it does NOT know:

- The specific format of spacecraft crash dump logs
- What specific fault codes mean in ECSS standards
- The causal relationships between specific telemetry parameters
- How to reason step-by-step through a spacecraft anomaly
- What a valid vs. dangerous recovery command looks like

Fine-tuning is the process of giving the model hundreds or thousands of examples of spacecraft fault scenarios paired with expert analysis and recovery steps, so it learns this domain-specific knowledge on top of its general intelligence.

Think of it like this: you hire a brilliant MIT computer science graduate (the pre-trained model) who has never worked on satellites. Fine-tuning is the 6-month training program that teaches them to be a spacecraft fault analyst.

4.2 Input Parameters Used for Fine-Tuning (What You Feed the Model)

Your training examples should be structured as input/output pairs. Each input is a formatted representation of a spacecraft anomaly event. Here are the parameter groups included in the input:

Group A: Pre-Safe-Mode Telemetry Window (The 5 minutes before fault)

This is the most informative section — the events leading up to the failure. You extract:

- Timestamps of each anomalous reading relative to safe mode entry (T-300s, T-60s, T-10s, T-0)
- Each affected parameter name, its nominal range, and the observed anomalous value
- Rate of change — did the value drift slowly or jump suddenly?
- Which subsystem each parameter belongs to
- Cross-parameter correlations — which parameters changed at the same time

Group B: Fault Register State (The Crash Dump Core)

A bitmask register where each bit represents a specific fault condition. For example:

```
FAULT_REG: 0x00000142 → Bit 1=SET (Battery undervolt), Bit 6=SET (Watchdog timeout), Bit 8=SET (Attitude error)
```

Your model must learn to decode these bitmasks and understand what combination of bits means what kind of failure.

Group C: Event Log Sequence (The Timeline)

A timestamped sequence of software events. For example:

```
T-00:04:21 GYRO_A_HEALTH_MONITOR: gyro_rate = NaN (fault threshold exceeded)
```

```
T-00:04:22 ADCS_CONTROLLER: attitude_error = 7.3 deg (nominal < 0.01)
```

```
T-00:04:23 SAFE_MODE_ENTRY: triggered by ADCS_ERROR_THRESHOLD
```

Group D: Hardware State Registers

Raw register values from the flight computer at the moment of safe mode entry: last reset cause register, SEU event counter since last boot, software process table (which tasks were running), and memory allocation map (looking for leaks).

Group E: Operating Context

Environmental context at fault time: orbital position (eclipse fraction, sun angle), mission phase (nominal science, maneuver, commissioning), time since last ground contact, and time since last safe mode entry.

4.3 Output Labels (What You Train the Model to Produce)

Each training example is paired with a structured output. This is the expert analysis that becomes the ground truth your model learns to replicate:

Output Field	Description
Root Cause Classification	One of: EPS_POWER_FAULT, ADCS_SENSOR_FAULT, OBC_SOFTWARE_FAULT, TCS_THERMAL_FAULT, COMMS_FAULT, MULTI_SYSTEM_CASCADE
Root Cause Subsystem	The specific subsystem (e.g., GYRO_A, REACTION_WHEEL_2, SOLAR_ARRAY_B)
Causal Chain	Ordered list: ROOT_EVENT → PROPAGATION_1 → PROPAGATION_2 → SAFE_MODE_TRIGGER
Confidence Score (0–1)	How certain is the diagnosis. Below 0.7 → flag for human review
Recovery Action Sequence	Ordered list of safe recovery commands with wait-and-verify steps between each
Risk Level of Recovery	LOW (safe to auto-execute) / MEDIUM (needs verification) / HIGH (escalate to human)
Similar Historical Incidents	References to past anomaly cases this resembles (for RAG retrieval)

4.4 The Critical Relationships the Model Must Learn

This section explains — in simple terms — the causal physics your model needs to internalize about how spacecraft parameters are related:

⚡ Power Chain Relationship

Solar Array generates electricity → stored in Battery → distributed by Power Regulator → consumed by subsystems. If any link breaks: I_{sa} drops → V_{bat} drops → V_{bus} drops → power shed → safe mode. Your model must learn: 'a sudden drop in solar current during sunlight always means array problem, not battery problem.'

🔄 Attitude-Power-Thermal Triangle

These three systems are deeply linked. If attitude control fails → satellite tumbles → solar panels lose sun pointing → power drops → heaters go off → thermal limits exceeded → hardware damage risk. Your model must know these cascade sequences cold.

💻 Software Fault Signature

Stuck CPU (100% load) + watchdog overflow + memory monotonically increasing → all three together strongly predict a software infinite loop. None alone is conclusive; all three together is a near-certain diagnosis. Your model learns these multi-parameter 'signatures'.

☢️ SEU Signature

A sudden spike in the SEU counter (bit flip events from cosmic rays) followed immediately by anomalous readings in ONE specific subsystem strongly suggests a radiation-induced fault in that subsystem — not a hardware failure. The recovery action is different: restart the subsystem software rather than replacing hardware.

SECTION 5 — MODEL SELECTION & FINE-TUNING APPROACH

5.1 Recommended Models (With Reasoning)

You want a model that is: (a) small enough to run on a laptop or small edge server, (b) capable of multi-step reasoning, and (c) fine-tunable with limited data and hardware. Here are the top recommendations:

Model	Hackathon Rating	Why / Why Not
Microsoft Phi-3-mini (3.8B params)	★★★★★ BEST CHOICE	Exceptional reasoning for its size. Runs on 8GB RAM at 4-bit quantization. Specifically trained for reasoning tasks. Ideal for structured JSON output (crash dump parsing).
Meta Llama-3.2-3B (3B params)	★★★★☆ STRONG CHOICE	Open weights, excellent community support, strong instruction following. Slightly less reasoning depth than Phi-3 but more customizable.
Microsoft Phi-3.5-mini (3.8B)	★★★★★ ALSO EXCELLENT	Newer than Phi-3-mini, improved on multi-step reasoning and tool use. Best overall if you want latest model.
Google Gemma-2-2B (2B params)	★★★☆☆ LIGHTWEIGHT	Smallest option that still reasons well. Use if hardware is very constrained. Less suited for complex causal reasoning.
Mistral-7B-Instruct (7B params)	★★★★☆ POWERFUL BUT HEAVIER	Better reasoning capacity but needs 16GB RAM for 4-bit inference. Use only if you have a GPU with ≥8GB VRAM.
IBM Granite-3.1-2B (2B params)	★★★☆☆ NICHE OPTION	IBM's enterprise-focused open model. Good for structured data reasoning. Worth testing as an alternative.

RECOMMENDATION FOR YOUR HACKATHON: Start with Phi-3.5-mini or Phi-3-mini. Run it with 4-bit GGUF quantization via llama.cpp. This requires only 4–6 GB RAM and runs on any modern laptop, no GPU needed.

5.2 What is QLoRA and Why You Use It

Full fine-tuning means updating ALL parameters of the model — for a 3.8 billion parameter model, this would require a data centre. QLoRA (Quantized Low-Rank Adaptation) is a clever trick that makes fine-tuning possible on a single consumer GPU.

Here is the intuition: instead of rewriting the entire model, you add tiny trainable "adapter" layers at strategic points. These adapters learn your domain-specific knowledge. The rest of the model is frozen (not updated) and compressed to 4-bit precision to save memory. The result: you train only about 0.1% of the total parameters, using about 6–8 GB of VRAM instead of 200+ GB.

Method	Details
Full fine-tuning	Update all 3.8B parameters Needs ~80 GB VRAM Requires A100 GPU cluster NOT feasible for hackathon
QLoRA fine-tuning (YOUR APPROACH)	Update only ~4M adapter parameters Needs ~8 GB VRAM (or CPU only for tiny datasets) Runs on RTX 3080 or even M2 MacBook Pro PERFECT for hackathon

5.3 Training Data Requirements

Here is how many training examples you need and where to get them:

For the Hackathon (Minimum Viable Dataset)

- 50–200 synthetic fault scenarios covering all 6 subsystem fault types
- Each example: input (structured crash dump JSON) + output (diagnosis + recovery plan)
- Generated using your Python fault simulator (details in Section 7)
- Augmented with 20–30 real incident reports from NASA NTRS (public database of mission anomaly reports)

For Production (What the Final System Needs)

- 2,000–10,000 labeled fault scenarios across all subsystem types and failure modes
- Real ESA-ADB telemetry data for the anomaly detection layer
- ECSS engineering standards ingested as RAG knowledge base
- NASA Mission Operation Reports (MORs) — contain real anomaly descriptions and resolutions

5.4 Training Data Format (What Each Example Looks Like)

Your training data is formatted as JSON Lines (.jsonl), where each line is one training example:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a spacecraft fault diagnosis expert..."
    },
    {
      "role": "user",
      "content": "CRASH DUMP REPORT\nTimestamp: 2026-01-15T03:42:11Z\nFault Register: 0x00000042\nPre-fault Telemetry:\n  T-60s: GYRO_A_RATE = NaN (expected: 0.02 deg/s)\n  T-60s: ATTITUDE_ERROR = 7.3 deg (expected: <0.01 deg)\n  T-10s: REACTION_WHEEL_1_SPEED = 0 rpm (expected: 2400 rpm)\n..."
    },
    {
      "role": "assistant",
      "content": "ROOT CAUSE: ADCS_SENSOR_FAULT\nAFFECTED_COMPONENT: GYRO_A\nCONFIDENCE: 0.91\nCAUSAL_CHAIN: GYRO_A returns NaN (probable SEU) → attitude knowledge lost → attitude_error exceeds threshold → ADCS cannot maintain pointing → safe mode triggered\nRECOVERY_PLAN:\n1. Verify SEU_COUNTER spike in OBC registers\n2. Execute: CMD_GYRO_A_RESET\n3. Wait 30s, verify GYRO_A_RATE returns valid reading\n4. If success: CMD_ATTITUDE_REACQUISITION\n5. If fail: switch to GYRO_B (backup), repeat steps..."
    }
  ]
}
```

5.5 Key Libraries for Fine-Tuning

Library	Purpose	Install Command
Hugging Face Transformers	Model loading, tokenization, training loop	pip install transformers
PEFT (Parameter Efficient Fine-Tuning)	Implements LoRA and QLoRA adapters	pip install peft
BitsAndBytes	4-bit/8-bit quantization for model compression	pip install bitsandbytes
Unsloth	2–5x faster QLoRA, lower VRAM usage — USE THIS	pip install unsloth
TRL (Transformer RL Library)	SFTTrainer — supervised fine-tuning wrapper	pip install trl
Datasets (HuggingFace)	Loading and managing your .jsonl training data	pip install datasets
Weights & Biases	Track training loss, visualize metrics	pip install wandb
LangGraph	Build the ReAct agent loop that uses your model	pip install langgraph
LlamaIndex	RAG pipeline over ECSS documentation	pip install llama-index
FAISS	Vector store for fast semantic document search	pip install faiss-cpu
DoWhy	Causal inference and fault tree construction	pip install dowhy

Skyfield	Orbital mechanics — satellite position calculations	pip install skyfield
----------	---	----------------------

SECTION 6 — COMPLETE AI PIPELINE

6.1 System Architecture Overview

Your system has five stages that execute sequentially whenever safe mode is detected. Think of it as an automated spacecraft doctor:

Stage	What Happens	Technology Used
Stage 1 Ingestion	Raw crash dump arrives. Parser converts hex registers, event log lines, and telemetry CSV into structured JSON that the AI can read.	Python custom parser, CCSDS decoder
Stage 2 Anomaly Filtering	LSTM/Autoencoder analyzes the 5-minute pre-fault telemetry window. Identifies which 5–20 parameters (out of 32,000) were actually anomalous. Narrows the haystack.	PyTorch, ESA-ADB dataset, trained LSTM
Stage 3 LLM Reasoning Agent	Fine-tuned Phi-3-mini reads the filtered anomaly report. Uses ReAct (Reason+Act) loop to iteratively hypothesize, query additional telemetry, and build a causal chain.	LangGraph, Phi-3-mini GGUF, llama.cpp
Stage 4 Knowledge Retrieval	As the agent reasons, it retrieves relevant documentation chunks via RAG. 'What does ECSS say about gyro reset procedures?' returns the exact specification page.	LlamaIndex, FAISS, ECSS PDFs as knowledge base
Stage 5 Recovery Execution	Agent generates recovery command sequence. Safety validator checks each command against whitelist. Commands issued incrementally with verify-then-proceed logic.	LangGraph tool nodes, rule-based safety gate

6.2 The ReAct Agent Loop — How the AI Reasons

ReAct is an AI reasoning pattern where the model alternates between Reasoning (thinking) and Acting (using tools). This is perfect for spacecraft diagnosis because it mirrors how an expert engineer actually thinks:

1. **THOUGHT:** 'The crash dump shows GYRO_A returned NaN at T-60s. Let me check what the SEU counter shows at that timestamp.'
2. **ACTION:** `query_telemetry_channel('SEU_COUNTER', 'T-120s to T-0s')`
3. **OBSERVATION:** 'SEU_COUNTER spiked from 0 to 3 events at T-62s — two seconds before GYRO_A faulted.'

4. THOUGHT: 'This is strongly consistent with a cosmic ray hit causing a bit flip in the gyro controller. Let me check what ECSS says about SEU-induced gyro failures.'
5. ACTION: `retrieve_procedure('SEU_induced_sensor_fault_ADCS')`
6. OBSERVATION: [Returns ECSS document section on single-event upset recovery for ADCS sensors]
7. THOUGHT: 'The standard procedure is: reset the sensor driver → verify reading → switch to backup if unrecovered. I have enough confidence to propose this recovery sequence. Risk level: LOW.'
8. OUTPUT: Root cause = SEU in GYRO_A controller. Recovery Plan = [CMD_GYRO_A_DRIVER_RESET, wait 30s, verify, if fail: CMD_SWITCH_TO_GYRO_B]

6.3 Model Performance Verification — How to Know Your Model Works

This is critical. A model that seems to work but is actually unreliable could issue dangerous recovery commands to a satellite. You need rigorous evaluation. Here is your complete evaluation framework:

Evaluation Set Construction

Hold out 20% of your synthetic fault scenarios as a test set (never shown during training). Also create a "hard" evaluation set with:

- Multi-system cascade failures (not single-subsystem)
- Fault types underrepresented in training data
- Ambiguous scenarios where two root causes are plausible
- Novel fault combinations the model has never seen

Metrics to Track

Metric	What It Measures & Target
Root Cause Classification Accuracy (%)	What % of faults did the model identify the correct subsystem? Target: > 80% Good: > 90%
Causal Chain F1 Score	How well does the model's causal chain match the ground truth? Use token-level F1 on the chain text. Target: > 0.75
Recovery Command Precision (%)	What % of proposed recovery commands are correct? Target: 100% (zero dangerous commands) Any unsafe command = immediate model rejection
Hallucination Rate (%)	How often does the model invent fake fault codes or parameters? Target: < 5% Measured by checking outputs against parameter name whitelist
Reasoning Chain Coherence	Is the logic actually correct? Use LLM-as-Judge evaluation (Claude as evaluator) Target: > 85% rated as coherent
Time-to-Diagnosis	How many seconds does the model take to produce a diagnosis? Target: < 60 seconds Baseline (human): 1–3 days

Confidence Calibration	When the model says 0.9 confidence, is it right 90% of the time? Measured with reliability diagrams Uncalibrated model = overconfident = dangerous
-------------------------------	---

Ablation Study (Proving Each Component Adds Value)

An ablation study removes each component of your system and measures the performance drop. This is the scientific way to prove your design choices are correct:

Configuration	Expected Performance
Full system (baseline)	F1=0.91, Accuracy=89%, Hallucination=3%
No anomaly filter (raw input to LLM)	F1=0.74 (-17pts) — model overwhelmed by full parameter set
No RAG knowledge base	F1=0.71 (-20pts) — model invents procedures not in standards
No QLoRA fine-tuning (base model only)	F1=0.53 (-38pts) — base model struggles with domain-specific reasoning
No causal graph (direct output)	F1=0.83 (-8pts) — slightly worse explainability and multi-hop reasoning

SECTION 7 — DATASETS

7.1 Dataset Overview

Dataset	What It Contains & Use	Access
ESA-ADB (Anomaly Detection Benchmark)	31 GB of REAL telemetry from 3 ESA missions. Labeled anomalies. YOUR PRIMARY DATASET for the anomaly detection layer (Stage 2).	Free, CC BY 3.0 doi.org/10.5281/zenodo.12528696
NASA SMAP / MSL	Telemetry from NASA's soil moisture satellite and Mars rover. 82 channels, 105 labeled anomaly sequences. Classic benchmark for spacecraft anomaly detection.	Free, public github.com/nasa-jpl/teleanom
CATS Dataset	2 GB, 17 variables from complex simulated system. 200 injected anomalies. Good for fault injection methodology reference.	Free, public Zenodo CATS dataset
ECSS Standards (PDF)	All spacecraft engineering and FDIR standards. YOUR PRIMARY RAG KNOWLEDGE BASE for the LLM agent. ECSS-E-ST-70, ECSS-Q-ST-30, and related.	Free, public ecss.nl
NASA NTRS Anomaly Reports	Mission Operations Reports — real anomaly descriptions with resolutions. Use as examples for fine-tuning and as RAG knowledge. Search NTRS for 'anomaly', 'safe mode', 'FDIR'.	Free, public ntrs.nasa.gov
Synthetic Fault Corpus (YOU GENERATE THIS)	50–500 synthetic crash dumps covering all fault types.	You generate this using Python fault simulator

	Paired with expert diagnosis + recovery plans.	(see Section 7.2 below)
	This IS a novel contribution — no public equivalent exists.	

7.2 How to Generate Your Synthetic Dataset

Since no public crash dump + diagnosis dataset exists, you will generate one. This is itself a research contribution. Here is the conceptual approach:

1. DEFINE FAULT SCENARIOS: Create a catalogue of fault scenarios covering each subsystem (EPS, ADCS, OBC, TCS, COMMS). For each scenario, define: the root cause, which parameters go anomalous, in what sequence, and what the correct diagnosis and recovery is.
2. BUILD A PARAMETER SIMULATOR: Write a Python class that simulates nominal telemetry for each subsystem (parameterised with realistic ranges). This produces baseline 'healthy' data.
3. INJECT FAULTS: At a random time T, inject the fault by modifying parameter outputs: gyro rate → NaN, battery voltage → slow drift down, CPU load → spike to 100%. Simulate how the fault propagates to other parameters over time.
4. GENERATE CRASH DUMP: At T+safe_mode, capture all parameter states, fault register bitmasks, and the event log. Format exactly as a real crash dump would look.
5. WRITE GROUND TRUTH LABELS: For each generated scenario, write the correct diagnosis (root cause, causal chain, recovery steps) as the expected LLM output.
6. AUGMENT WITH NOISE: Add realistic sensor noise, occasional NaN values, and unit conversion errors to make the data more realistic.

SECTION 8 — NOVELTY & INNOVATIONS THAT MAKE THIS STAND OUT

8.1 Core Novel Claims

Your system has THREE genuinely novel contributions that do not exist in any published work:

NOVELTY 1: First LLM-based autonomous safe mode recovery agent

While arxiv:2404.00413 proved LLMs can control satellites in simulation, and ESA's orbital_OLIVER (2023) achieved TRL 4 for anomaly detection, NO published system combines LLM reasoning with full safe mode recovery command execution. You are filling an explicitly acknowledged gap in ESA's own specifications.

NOVELTY 2: First synthetic spacecraft crash dump corpus with labeled diagnoses

No public dataset of spacecraft crash dumps paired with expert diagnoses exists. By generating and releasing your synthetic corpus, you provide the research community with a training resource that has never existed — a standalone scientific contribution.

NOVELTY 3: Causal graph visualization of spacecraft fault propagation

Representing the fault propagation path as an interactive directed acyclic graph (DAG) — not just a text diagnosis — makes the AI's reasoning auditable. Engineers can review exactly why the model proposed a specific recovery action, dramatically increasing trustworthiness for actual deployment.

8.2 Additional Innovations to Maximise Impact

A: Multi-hypothesis ranking with confidence scores

Instead of committing to a single diagnosis, the agent produces a ranked list of 3 hypotheses with confidence scores. If the top hypothesis recovery fails, it automatically tries the second. This mirrors how expert engineers actually work and makes the system much more robust.

B: Anomaly onset detection as early warning

Your LSTM pre-filter (Stage 2) does not just detect anomalies at safe mode time — it can be run continuously and flag 'pre-anomaly patterns' that precede safe mode by 30–120 minutes. This enables PREDICTIVE safe mode prevention, a step beyond recovery. Judges will love this.

C: Recovery command safety sandbox

Before any recovery command is executed, it passes through a formal whitelist + constraint checker. Commands that would: exceed attitude rate limits, discharge battery below survival threshold, or operate an instrument in a thermally unsafe state are automatically blocked. This safety layer is what makes real-world deployment conceivable.

D: Continuous learning from resolved incidents

Each time the system successfully diagnoses and recovers from a fault, the incident (input crash dump + recovery actions + outcome) is added to a growing knowledge base. Over time, the system's RAG knowledge base becomes richer, and the model can be periodically fine-tuned on the growing dataset of real resolved incidents.

SECTION 9 — HACKATHON EXECUTION ROADMAP

9.1 36-Hour Sprint Plan

Phase	Deliverable	Owner
Hours 0–3 Data Foundation	Generate 50 synthetic fault scenarios in Python (3 fault types: gyro SEU, power undervolt, software watchdog) Format as training JSONL Validate JSON schema	1 data engineer
Hours 3–8 RAG Knowledge Base	Download ECSS standards PDFs Chunk + embed using LlamaIndex Set up ChromaDB vector store Verify retrieval works with 5 test queries	1 ML engineer
Hours 8–18 Agent Core	Set up Phi-3-mini with llama.cpp (GGUF) Build LangGraph ReAct agent with 4 tools: query_telemetry, retrieve_procedure, check_status, propose_recovery Test on 10 synthetic fault scenarios	2 engineers
Hours 18–24 Fine-Tuning (optional)	If time allows: QLoRA fine-tune Phi-3-mini on synthetic dataset using Unsloth Run 2–3 epochs Evaluate accuracy improvement Export merged model	1 ML engineer
Hours 24–30 Demo Interface	Build Streamlit UI: crash dump input panel, agent reasoning trace panel, causal graph (NetworkX → matplotlib), recovery plan output panel Connect to agent backend	1 frontend
Hours 30–34 Evaluation	Run full evaluation suite on 20 held-out test scenarios Calculate accuracy, precision, and hallucination rate	All

	Document results for pitch	
Hours 34–36 Pitch Prep	Rehearse demo: Hubble story → manual process → demo live → results → novelty claims → future vision Prepare 3 backup static screenshots in case live demo fails	All

9.2 Minimum Viable Demo (If Everything Goes Wrong)

If the live fine-tuning fails or the agent is unstable, you can still demo the architecture convincingly:

- Show the synthetic crash dump generator producing realistic data
- Show the RAG system retrieving the correct ECSS procedure for a given fault query
- Use the base Phi-3-mini (no fine-tuning) with a well-crafted system prompt as your agent
- Pre-compute and show the causal graph for 3 pre-set scenarios
- Show the evaluation metrics on your test set from local runs

Winning Pitch Line

Open with: 'ESA's own specification says — Safe mode recovery requires ground interaction. No auto-recovery from safe mode is foreseen. That sentence was written to describe a limitation. We built the system that makes it obsolete.'

APPENDIX — QUICK REFERENCE GLOSSARY

Term	Definition
ADCS	Attitude Determination and Control System — the satellite's 'steering'
CCSDS	Consultative Committee for Space Data Systems — international standard for spacecraft data formats
DoWhy	Microsoft's open-source causal inference Python library
EPS	Electrical Power System — solar panels, batteries, power distribution
ESA-ADB	ESA Anomaly Detection Benchmark — 31 GB real satellite telemetry dataset
FDIR	Fault Detection, Isolation and Recovery — the classical onboard fault management system
GGUF	GPT-Generated Unified Format — efficient format for running quantized LLMs locally
IMU	Inertial Measurement Unit — measures angular velocity and acceleration, used for attitude control
LangGraph	LangChain's framework for building stateful multi-step AI agent workflows
LlamaIndex	Python library for building RAG (Retrieval-Augmented Generation) systems
OBC	On-Board Computer — the satellite's main flight computer

QLoRA	Quantized Low-Rank Adaptation — memory-efficient LLM fine-tuning technique
RAG	Retrieval-Augmented Generation — giving an LLM real-time access to a document knowledge base
ReAct	Reason + Act — AI agent design pattern alternating between thinking and tool use
SEU	Single Event Upset — a bit flip in computer memory caused by a cosmic ray particle
TCS	Thermal Control System — heaters and radiators that manage spacecraft temperature
Telemetry	Stream of sensor data the spacecraft continuously transmits to Earth
TRL	Technology Readiness Level — NASA scale 1–9 measuring maturity of a technology (9 = flight proven)
Watchdog Timer	A hardware/software timer that resets periodically; if it overflows, the computer restarts itself

— END OF DOCUMENT —